Mini Lab: Catching a Custom Exception

Bank account withdrawal in C#

2nd-year Computer Science – .NET Technology – Duration: about 45 minutes
Tools: Visual Studio or Visual Studio Code, .NET 8 console application

Learning objectives

By the end of this lab you will be able to:

- create a `List<T>` of objects and initialise it with random values;
- write a **custom exception** class derived from `Exception`;
- **throw** that exception from a business method;
- **catch** it with `try / catch / finally` and display a clear message.

1. Scenario

A small bank stores five accounts in memory. Each account has an identifier, a customer name and a balance chosen randomly between **5 000** and **100 000**. The program asks for an account number and an amount to withdraw. If the balance is lower than the requested amount, the withdrawal must be refused by throwing a custom exception whose message is:

```
Insufficient bank account balance
```

2. Project setup

Visual Studio
File → New → Project → Console App (.NET 8), name it `BankLab`.

VS Code
In a terminal: `dotnet new console -n BankLab`, then `cd BankLab` and `code .`

Run the program with `Ctrl+F5` (Visual Studio) or `dotnet run` (VS Code).

3. Step-by-step tasks

Task 1 – The custom exception

Create a class `InsufficientFundsException` that derives from `Exception`.

- Provide the three standard constructors (empty, `message`, `message + inner`).
- Add two read-only properties: `Balance` and `Requested` (both `decimal`).
- Add a constructor `(decimal balance, decimal requested)` whose message is `"Insufficient bank account balance"`.

Task 2 – The `BankAccount` class

Create a class with the properties `Id` (int), `CustomerName` (string) and `Balance` (decimal), and a method `Withdraw(decimal amount)` that:

1. throws `ArgumentOutOfRangeException` if `amount <= 0`;

2. throws `InsufficientFundsException` if `amount > Balance`;
3. otherwise subtracts the amount from the balance.

Task 3 – Create the five accounts

In `Program.cs`, build a `List<BankAccount>` of 5 accounts. Use one `Random` object and `rnd.Next(5000, 100001)` for each balance (the upper bound is exclusive). Display the list.

Task 4 – Withdraw and catch

Read an account id and an amount from the console (use `int.TryParse` / `decimal.TryParse`). Find the account with `accounts.Find(...)`. Call `Withdraw` inside a `try` block and:

- catch `InsufficientFundsException` and print its message, the balance and the requested amount;
- catch `ArgumentOutOfRangeException` for invalid amounts;
- use a `finally` block to print `"Operation finished."`

Hints

- `Find` returns `null` when no account matches: handle that case before calling `Withdraw`.
- Put the **specific** catch blocks first and do not leave a `catch` block empty.
- Use `decimal` for money, not `double`.

4. Starter code

Complete every `TODO`.

```
using System;
using System.Collections.Generic;

// ---------- Task 1 ----------
public class InsufficientFundsException : Exception
{
    // TODO: properties Balance and Requested (get only)
    // TODO: the three standard constructors
    // TODO: constructor (decimal balance, decimal requested)
    //       message = "Insufficient bank account balance"
}

// ---------- Task 2 ----------
public class BankAccount
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public decimal Balance { get; set; }

    public void Withdraw(decimal amount)
    {
        // TODO: amount <= 0  -> throw ArgumentOutOfRangeException
        // TODO: amount > Balance -> throw InsufficientFundsException
        // TODO: otherwise subtract the amount
    }
}

// ---------- Tasks 3 and 4 : Program.cs ----------
var rnd = new Random();
```

```
30  string[] names = { "Ada", "Linus", "Grace", "Alan", "Margaret" };
31  var accounts = new List<BankAccount>();
32
33  // TODO (Task 3): create 5 accounts with Id = 1..5 and a random balance
34  //                between 5000 and 100000, then display them
35
36  // TODO (Task 4): read id and amount, find the account,
37  //                try { Withdraw } catch { ... } finally { ... }
```

5. Expected output (example)

Balances are random, so your numbers will differ.

```
Id  Customer   Balance
1   Ada        42350
2   Linus      7800
3   Grace      95120
4   Alan       18400
5   Margaret   63010

Account id: 2
Amount to withdraw: 10000
ERROR: Insufficient bank account balance
       Balance = 7800, requested = 10000
Operation finished.
```

And with a valid operation:

```
Account id: 3
Amount to withdraw: 5000
Withdrawal OK. New balance = 90120
Operation finished.
```

6. Test checklist

Test	Input	Expected result
Valid withdrawal	amount $\leq$ balance	New balance displayed
Amount too large	amount $>$ balance	`Insufficient bank account balance`
Negative or zero amount	`-50` or `0`	Invalid amount message
Unknown account	id `99`	“Account not found”
Not a number	`abc`	“Invalid input”, no crash

7. Going further (optional)

- Add a `Deposit` method and a `Transfer(from, to, amount)` method that reuses `Withdraw`.
- Put the withdrawal in a loop until the user types `q`.
- Observe in the debugger (Exception Settings) where the exception is thrown and where it is caught.
- Write the failed operations to a file using `using` and `StreamWriter`.